

EXPERIMENT 1

The screenshot shows a Python IDE with two windows. The left window, titled 'IDLE Shell 3.12.10', displays the output of a program that solves the N-Queens problem for N=4, 5, and 8. The output shows the board configurations for each N value. The right window, titled '4.py - E:/daa lab/4.py (3.12.10)', contains the Python code for the backtracking algorithm. The code defines a 'safe' function to check if a queen can be placed at a given position, a 'backtrack' function to recursively find solutions, and a main loop that prints the solutions for N=4, 5, and 8.

```
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: E:/daa lab/4.py =====
N = 4
.Q..
...Q
Q...
..Q.

N = 5
Q....
..Q..
...Q.
.Q...
...Q.

N = 8
Q.....
...Q...
.....Q
...Q...
..Q....
.Q....
.Q....
..Q....

>>>
```

```
def safe(row, col):
    if col + (row - i) < n:
        if board[i][col + (row - i)] == 'Q':
            return False
    return True

def backtrack(row):
    if row == n:
        solutions.append([''.join(r) for r in board])
        return

    for col in range(n):
        if safe(row, col):
            board[row][col] = 'Q'
            backtrack(row + 1)
            board[row][col] = '.'

backtrack(0)
return solutions

for n in [4, 5, 8]:
    print("N =", n)
    solution = solve_n_queens(n)[0]

    for row in solution:
        print(row)

    print()
```

EXPERIMENT 2

The screenshot shows a Python IDE with two windows. The left window, titled 'IDLE Shell 3.12.10', displays the output of a program that solves the N-Queens problem for various board sizes and configurations. The output shows the board configurations for an 8x10 board, a 5x5 board with obstacles, and a 6x6 board. The right window, titled '4.py - E:/daa lab/4.py (3.12.10)', contains the Python code for the backtracking algorithm. The code defines a 'safe' function to check if a queen can be placed at a given position, a 'backtrack' function to recursively find solutions, and a main loop that prints the solutions for various board sizes and configurations.

```
ion.
>>>
===== RESTART: E:/daa lab/4.py =====
8 x 10 Board
['Q', '.', '.', '.', '.', '.', '.', '.', '.', '.']
['.', 'Q', '.', '.', '.', '.', '.', '.', '.', '.']
['.', '.', 'Q', '.', '.', '.', '.', '.', '.', '.']
['.', '.', '.', 'Q', '.', '.', '.', '.', '.', '.']
['.', '.', '.', '.', 'Q', '.', '.', '.', '.', '.']
['.', '.', '.', '.', '.', 'Q', '.', '.', '.', '.']
['.', '.', '.', '.', '.', '.', 'Q', '.', '.', '.']
['.', '.', '.', '.', '.', '.', '.', 'Q', '.', '.']

5 x 5 Board with Obstacles
['Q', '.', '.', '.', '.']
['.', 'Q', '.', '.', '.']
['.', '.', 'Q', '.', '.']
['.', '.', '.', 'Q', '.']
['.', '.', '.', '.', 'Q']

6 x 6 Board
['.', 'Q', '.', '.', '.', '.']
['.', 'Q', '.', '.', '.', '.']
['.', 'Q', '.', '.', '.', '.']
['.', 'Q', '.', '.', '.', '.']
['.', 'Q', '.', '.', '.', '.']
['.', 'Q', '.', '.', '.', '.']

>>>
```

```
def safe(row, col):
    if backtrack(row + 1):
        return True
    board[row][col] = '.'
    return False

backtrack(0)
return solution

print("8 x 10 Board")
board = solve_rectangular(8, 10)

for row in board:
    print(row)

print("\n5 x 5 Board with Obstacles")
board = solve_rectangular(5, 5, {(2, 2), (4, 4)})

for row in board:
    print(row)

print("\n6 x 6 Board")
board = solve_rectangular(6, 6, restricted={1, 3, 5})

for row in board:
    print(row)
```

EXPERIMENT 3

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: E:/daa lab/4.py =====
=====
['5', '3', '4', '6', '7', '8', '9', '1', '2']
['6', '7', '2', '1', '9', '5', '3', '4', '8']
['1', '9', '8', '3', '4', '2', '5', '6', '7']
['8', '5', '9', '7', '6', '1', '4', '2', '3']
['4', '2', '6', '8', '5', '3', '7', '9', '1']
['7', '1', '3', '9', '2', '4', '8', '5', '6']
['9', '6', '1', '5', '3', '7', '2', '8', '4']
['2', '8', '7', '4', '1', '9', '6', '3', '5']
['3', '4', '5', '2', '8', '6', '1', '7', '9']
>>>

4.py - E:/daa lab/4.py (3.12.10)
File Edit Format Run Options Window Help

for num in "123456789":

    if valid(row, col, num):
        board[row][col] = num

    if solve_sudoku(board):
        return True

    board[row][col] = '.'

return False

board = [
    ["5", "3", ".", ".", "7", ".", ".", ".", "."],
    ["6", ".", ".", "1", "9", "5", ".", ".", "."],
    [".", "9", "8", ".", ".", ".", ".", "6", "."],
    ["8", ".", ".", "6", ".", ".", ".", "3", "."],
    ["4", ".", ".", "8", ".", "3", ".", ".", "1"],
    ["7", ".", ".", "2", ".", ".", ".", "6", "."],
    [".", "6", ".", ".", ".", "2", "8", ".", "."],
    [".", ".", ".", "4", "1", "9", ".", ".", "5"],
    [".", ".", ".", ".", "8", ".", ".", "7", "9"]
]

solve_sudoku(board)

for row in board:
    print(row)
```

EXPERIMENT 4

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: E:/daa lab/4.py =====
=====
5
1
>>>

4.py - E:/daa lab/4.py (3.12.10)
File Edit Format Run Options Window Help

def findTargetSumWays(nums, target):

    count = 0

    def backtrack(index, total):
        nonlocal count

        if index == len(nums):
            if total == target:
                count += 1
            return

        backtrack(index + 1, total + nums[index])
        backtrack(index + 1, total - nums[index])

    backtrack(0, 0)

    return count

print(findTargetSumWays([1,1,1,1,1], 3))
print(findTargetSumWays([1], 1))
```

EXPERIMENT 5

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: E:/daa lab/4.py =====
17
444
>>>
```

```
def sumSubarrayMins(arr):
    if not arr:
        return 0
    stack = []
    for i in range(len(arr) - 1, -1, -1):
        while stack and arr[stack[-1]] >= arr[i]:
            stack.pop()
        if stack:
            left[i] = i - stack[-1]
        else:
            left[i] = i + 1
        stack.append(i)
    stack = []
    for i in range(len(arr)):
        while stack and arr[stack[-1]] >= arr[i]:
            stack.pop()
        if stack:
            right[i] = stack[-1] - i
        else:
            right[i] = len(arr) - i
        stack.append(i)
    total = 0
    for i in range(len(arr)):
        total += arr[i] * left[i] * right[i]
    return total % mod

print(sumSubarrayMins([3,1,2,4]))
print(sumSubarrayMins([11,81,94,43,3]))
```

EXPERIMENT 6

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: E:/daa lab/4.py =====
[[2, 2, 3], [7]]
[[2, 2, 2, 2], [2, 3, 3], [3, 5]]
>>>
```

```
def combinationSum(candidates, target):
    result = []
    candidates.sort()
    def backtrack(start, target, current):
        if target == 0:
            result.append(current[:])
            return
        for i in range(start, len(candidates)):
            if candidates[i] > target:
                break
            current.append(candidates[i])
            backtrack(i, target - candidates[i], current)
            current.pop()
        backtrack(0, target, [])
    return result

print(combinationSum([2,3,6,7], 7))
print(combinationSum([2,3,5], 8))
```

EXPERIMENT 7

The screenshot shows a Python IDE with two windows. The left window is the IDLE Shell, displaying the Python version (3.12.10) and the execution of a restart command. The right window shows a Python script for a combination sum problem using backtracking.

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: E:/daa lab/4.py =====
>>> [[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]
>>> [[1, 2, 2], [5]]

candidates.sort()

def backtrack(start, target, current):

    if target == 0:
        result.append(current[:])
        return

    for i in range(start, len(candidates)):

        if i > start and candidates[i] == candidates[i - 1]:
            continue

        if candidates[i] > target:
            break

        current.append(candidates[i])
        backtrack(i + 1, target - candidates[i], current)
        current.pop()

    backtrack(0, target, [])

    return result

print(combinationSum2([10,1,2,7,6,1,5], 8))
print(combinationSum2([2,5,2,1,2], 5))
```

EXPERIMENT 8

The screenshot shows a Python IDE with two windows. The left window is the IDLE Shell, displaying the Python version (3.12.10) and the execution of a restart command. The right window shows a Python script for a permutation problem using backtracking.

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: E:/daa lab/4.py =====
>>> [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
>>> [[0, 1], [1, 0]]
>>> [[1]]

def permute(nums):

    result = []

    def backtrack(current, remaining):

        if len(remaining) == 0:
            result.append(current[:])
            return

        for i in range(len(remaining)):

            current.append(remaining[i])

            backtrack(
                current,
                remaining[:i] + remaining[i+1:]
            )

            current.pop()

        backtrack([], nums)

    return result

print(permute([1,2,3]))
print(permute([0,1]))
print(permute([1]))
```

EXPERIMENT 9

```

Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: E:/daa lab/4.py =====
>>>
[[1, 1, 2], [1, 2, 1], [2, 1, 1]]
[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
>>>

def backtrack(current):
    if len(current) == len(nums):
        result.append(current[:])
        return

    for i in range(len(nums)):
        if used[i]:
            continue

        if i > 0 and nums[i] == nums[i-1] and not used[i-1]:
            continue

        used[i] = True
        current.append(nums[i])

        backtrack(current)

        current.pop()
        used[i] = False

    backtrack([])

    return result

print(permuteUnique([1,1,2]))
print(permuteUnique([1,2,3]))

```

EXPERIMENT 10

```

Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: E:/daa lab/4.py =====
>>>
(3, [0, 1, 2, 1])
>>>

def can_color(vertex, color):
    colors[vertex] = color

    if backtrack(vertex + 1, max_colors):
        return True

    colors[vertex] = -1

    return False

for color_count in range(1, n + 1):
    colors = [-1] * n

    if backtrack(0, color_count):
        return color_count, colors

edges = [
    (0,1),
    (1,2),
    (2,3),
    (3,0),
    (0,2)
]

print(graph_coloring(4, edges))

```

EXPERIMENT 11

The screenshot shows two windows from the IDLE 3.12.10 environment. The left window, titled 'IDLE Shell 3.12.10', displays the Python interpreter's startup message and a 'RESTART' prompt. The user has entered the list `[0, 1, 2, 1]`. The right window, titled '4.py - E:/daa lab/4.py (3.12.10)', contains the following Python code:

```
for color in range(k):  
    if safe(vertex, color):  
        colors[vertex] = color  
        if solve(vertex + 1):  
            return True  
        colors[vertex] = -1  
    return False  
if solve(0):  
    return colors  
return None  
  
edges = [  
    (0,1),  
    (1,2),  
    (2,3),  
    (3,0),  
    (0,2)  
]  
  
print(graph_coloring_k(4, edges, 3))
```

The taskbar at the bottom shows the system clock as 15:41 on 30-08-2026.

EXPERIMENT 12

The screenshot shows two windows from the IDLE 3.12.10 environment. The left window, titled 'IDLE Shell 3.12.10', displays the Python interpreter's startup message and a 'RESTART' prompt. The user has entered the tuple `(False, [])`. The right window, titled '4.py - E:/daa lab/4.py (3.12.10)', contains the following Python code:

```
visited.add(next_vertex)  
path.append(next_vertex)  
  
if backtrack():  
    return True  
  
path.pop()  
visited.remove(next_vertex)  
  
return False  
  
if backtrack():  
    return True, path + [0]  
return False, []  
  
edges = [  
    (0,1),  
    (1,2),  
    (2,3),  
    (3,0),  
    (0,2),  
    (2,4),  
    (4,0)  
]  
  
print(hamiltonian_cycle(5, edges))
```

The taskbar at the bottom shows the system clock as 15:42 on 30-08-2026.

EXPERIMENT 13

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: E:/daa lab/4.py =====
True
>>>

return path[0] in graph[path[-1]]

for v in graph[path[-1]]:
    if v not in visited:
        visited.add(v)
        path.append(v)

        if solve():
            return True

        path.pop()
        visited.remove(v)

    return False

return solve()

edges = [
    (0,1),
    (1,2),
    (2,3),
    (3,0),
    (0,2)
]

print(hamiltonian_cycle(4, edges))
```

EXPERIMENT 14

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: E:/daa lab/4.py =====
[[[]], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]
>>>

def subsets(nums):
    result = []

    def backtrack(index, current):
        if index == len(nums):
            result.append(current[:])
            return

        backtrack(index + 1, current)
        current.append(nums[index])
        backtrack(index + 1, current)
        current.pop()

    backtrack(0, [])
    result.sort()
    return result

A = [1,2,3]
print(subsets(A))
```

EXPERIMENT 15

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: E:/daa lab/4.py =====
[[3], [3, 5], [3, 4], [3, 4, 5], [2, 3], [2, 3, 5], [2, 3, 4], [2, 3, 4, 5]]
[[], [3], [2], [2, 3], [1], [1, 3], [1, 2], [1, 2, 3]]
[[], [0]]

>>>
```

```
4.py - E:/daa lab/4.py (3.12.10)
File Edit Format Run Options Window Help

print(subsets_containing(E, 3))

def power_set(nums):
    result = []

    def backtrack(index, current):
        if index == len(nums):
            result.append(current[:])
            return

        backtrack(index + 1, current)

        current.append(nums[index])

        backtrack(index + 1, current)

        current.pop()

    backtrack(0, [])

    return result

print(power_set([1,2,3]))
print(power_set([0]))
```

EXPERIMENT 16

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: E:/daa lab/4.py =====
['facebook', 'google', 'leetcode']
['apple', 'google', 'leetcode']

>>>
```

```
4.py - E:/daa lab/4.py (3.12.10)
File Edit Format Run Options Window Help

valid = True

for char in required:
    if count[char] < required[char]:
        valid = False
        break

if valid:
    result.append(word)

return result

words1 = [
    "amazon",
    "apple",
    "facebook",
    "google",
    "leetcode"
]

words2 = ["e", "o"]

print(wordSubsets(words1, words2))

words2 = ["l", "e"]

print(wordSubsets(words1, words2))
```